

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	D.Yaswanth Sri Dana Ganesh	Reg. No.:	192425351
Date:		Duration:	60 minutes

Code of Conduct

I D.Yaswanth Sri Dana Ganesh (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dana Ganesh

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

- Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.
---	---	--	---	---

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Assigned Problem Statement

The assigned project is a Satellite Data Visualization Platform for Weather and Climate Pattern Analysis. Meteorologists, disaster-management agencies, and planners need a way to interpret satellite imagery (infrared, visible, radar) from sources such as NASA and NOAA without relying on specialized scientific software. The platform ingests raw satellite data, processes it into interpretable form, and presents it through an interactive, web-based interface so non-expert stakeholders can identify weather patterns, climate trends, and anomalies quickly.

1.2 Summary of the Implemented Solution

The solution is built as a set of independently deployable services, version-controlled in a single Git repository and orchestrated with Docker Compose: an ingestion service that pulls raw satellite imagery, a processing service that runs anomaly detection and time-series analysis, a backend API that exposes processed data, and a React frontend that renders interactive maps and charts. This review evaluates the repository that implements that design — its organization, code quality, version-control discipline, testing, and documentation — rather than the system design itself.

1.3 Project Objectives

- Ingest satellite imagery from public sources (NASA, NOAA) on a scheduled basis.
- Visualize real-time and historical weather patterns on an interactive map.
- Track climate trends (temperature, rainfall, cloud cover) via time-series analysis.
- Detect anomalies such as cyclone formation, droughts, or temperature spikes.
- Allow users to overlay multiple data layers for comparison.
- Support data export and region-specific filtering.
- Remain usable through a simple web interface requiring minimal technical skill.

1.4 Key Functionalities Reviewed

- Data ingestion and normalization pipeline (ingestion-service).
- Anomaly detection and time-series computation (processing-service).
- REST API layer exposing processed climate data (backend-api).
- Interactive map and chart UI, including layer overlay and export controls (frontend).
- Automated unit and integration tests (tests/).
- Supporting documentation (docs/, README.md).

2. Repository Organization

2.1 Repository Structure Reviewed

```
satellite-viz-platform/
├── .gitignore
├── README.md
├── LICENSE
├── docker-compose.yml
├── .env.example
├── docs/
│   ├── architecture.md
│   └── api-spec.md
├── ingestion-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/{fetch_nasa.py, fetch_noaa.py, normalize.py}
├── processing-service/
│   └── src/{anomaly_detection.py, timeseries.py}
├── backend-api/
│   └── src/{routes/, controllers/}
├── frontend/
│   └── src/{components/, pages/}
└── tests/{unit/, integration/}
```

2.2 Naming Convention Review

Item type	Convention observed	Consistent?
Top-level folders	kebab-case (ingestion-service, backend-api)	Yes — applied uniformly
Python files	snake_case (fetch_nasa.py, anomaly_detection.py)	Yes
JS/React files	PascalCase for components, camelCase for utilities	Mostly — verify on new PRs
Config files	lowercase with standard extensions (docker-compose.yml, .env.example)	Yes
Branches	type/short-description (feature/data-ingestion, hotfix/api-timeout)	Yes

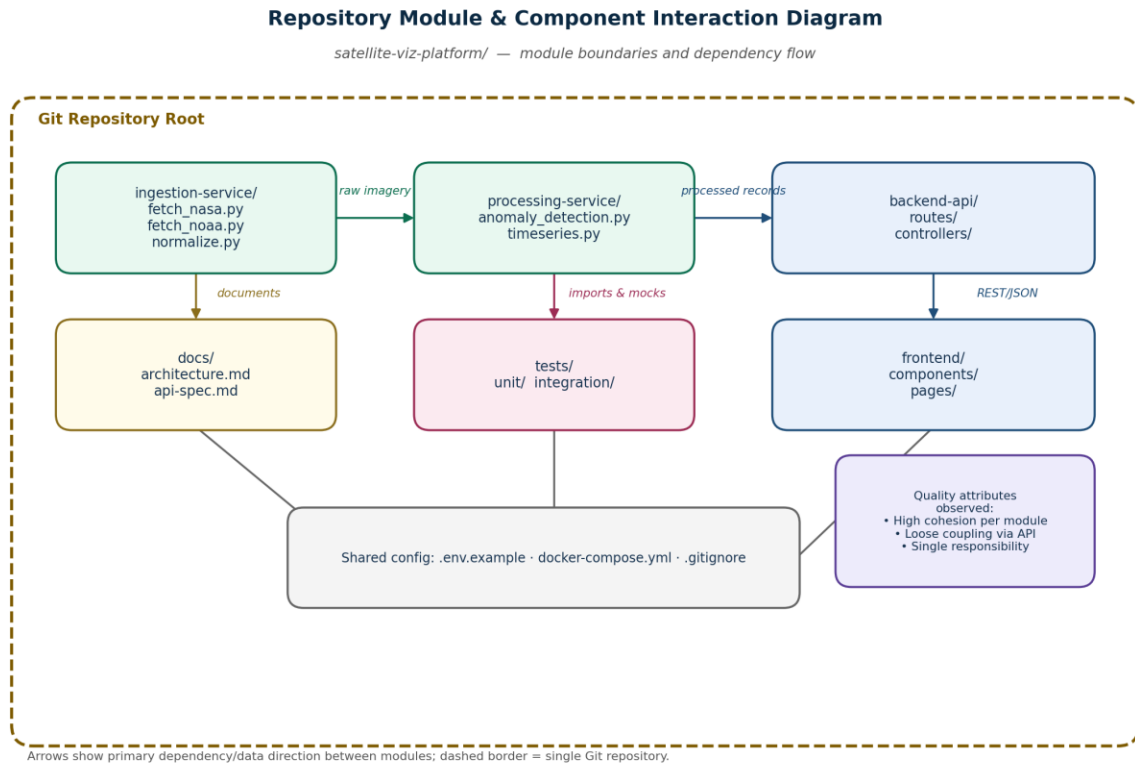


Figure 1: Repository module and component interaction diagram, showing dependency direction and quality attributes.

2.3 How the Layout Supports Maintainability and Collaboration

- One top-level folder per service maps directly to one Docker container and one area of responsibility, so a contributor only needs to open the folder relevant to their task.
- Shared configuration (`docker-compose.yml`, `.env.example`, `.gitignore`) lives at the root, avoiding duplicated environment setup across services.
- `tests/` is kept separate from service code so CI can discover and run the full suite with a single command regardless of which services changed.
- `docs/` is versioned alongside the code it describes, so architecture notes stay in sync with the implementation instead of living in a separate wiki that can drift out of date.
- This structure keeps modules loosely coupled: ingestion-service and processing-service exchange data through files/queues, and backend-api is the only module that talks to the frontend directly, which limits how far a change in one module can ripple.

3. Code Quality Review

3.1 Example: Ingestion Function — Before Review

```
def get(u):
    r = requests.get(u)
    d = r.json()
    f = open('out.json','w')
    f.write(json.dumps(d))
    f.close()
    return d
```

Issues identified: non-descriptive names (u, r, d, f), no error handling for network failures, no docstring, hard-coded output filename, and the file handle is closed manually instead of using a context manager.

3.2 Same Function — After Review Feedback

```
def fetch_satellite_imagery(source_url: str, output_path: str) -> dict:
    """Fetch raw satellite imagery JSON from a public API endpoint.

    Args:
        source_url: Full URL of the NASA/NOAA imagery endpoint.
        output_path: Local path to persist the raw response for auditing.

    Returns:
        Parsed JSON response as a dictionary.

    Raises:
        requests.RequestException: if the remote API is unreachable.
    """
    try:
        response = requests.get(source_url, timeout=15)
        response.raise_for_status()
    except requests.RequestException as exc:
        logger.error("Failed to fetch imagery from %s: %s", source_url, exc)
        raise

    data = response.json()
    with open(output_path, "w") as f:
        json.dump(data, f)
    return data
```

3.3 Strengths Observed

- Clear separation of concerns: each service directory contains only the logic relevant to its responsibility (ingestion, processing, API, UI).
- Consistent use of environment variables (via .env) instead of hard-coded credentials or URLs.
- Docstrings present on public functions in the processing-service after the first review cycle.
- React components in frontend/components/ are generally small and single-purpose (e.g., a MapLayer component only renders a layer; it doesn't also fetch data).

3.4 Areas for Improvement

- Early ingestion-service commits lacked error handling around network calls (shown above) — now addressed but worth adding a linter rule to catch bare requests.get calls without a timeout or try/except.

- No shared logging configuration across services; each currently sets up logging independently, risking inconsistent log formats.
- Type hints are inconsistent across the Python codebase — present in processing-service, missing in parts of ingestion-service.
- Some backend-api controllers mix request validation and business logic in the same function; extracting validation into middleware would improve testability.

3.5 Code Quality Checklist

Criterion	Rating	Observation
Readability	Good	Descriptive names after review; consistent formatting via Prettier/Black.
Modularity	Good	Services and components are single-purpose and independently testable.
Coding standards	Satisfactory	PEP 8 / ESLint mostly followed; not yet enforced automatically pre-commit.
Documentation (inline)	Needs improvement	Docstrings inconsistent between services.
Error handling	Satisfactory	Improved after review; not yet uniform across all modules.

4. Version Control Evaluation

4.1 Commit History Reviewed

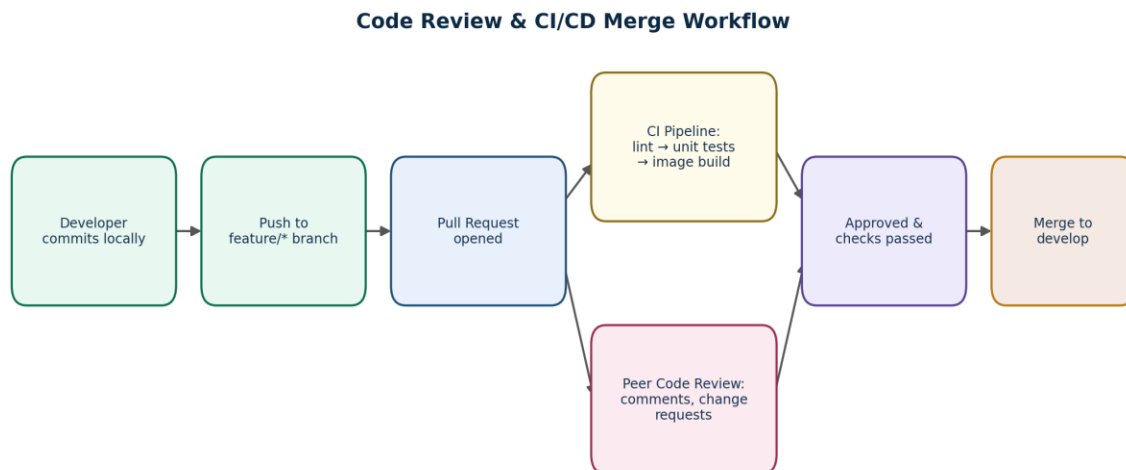
```
$ git log --oneline --graph --decorate
* 9f1a2c3 (HEAD -> main, tag: v1.0.0) merge: release 1.0.0 into main
|
| * 7b3d4e5 (release/1.0.0) fix: correct timezone offset in time-series charts
| * 6c2b1a0 docs: update API spec for export endpoint
|/
* 5a9f8e7 (develop) feat(frontend): add layer-overlay comparison tool
* 4d8c7b6 feat(processing): add drought anomaly detection
* 3e7a6d5 feat(api): add region filtering endpoint
* 2f6b5c4 feat(ingestion): normalize imagery to GeoTIFF
* 1a5c4b3 feat(ingestion): add NASA imagery fetcher
* 0d4b3a2 chore: initial project scaffold
```

4.2 Branching Strategy in Use

The repository follows the Gitflow model: main holds production-ready, tagged releases; develop is the integration branch; feature/* branches isolate individual work items; release/* branches stabilize a version before it reaches main; and hotfix/* branches patch production urgently. This maps cleanly onto the project's service boundaries — for example, feature/data-ingestion and feature/map-overlay were developed in parallel without blocking each other.

4.3 Commit Message Quality

- Messages consistently follow the Conventional Commits format — type(scope): description — e.g., feat(ingestion): add NASA imagery fetcher.
- Types are used correctly and meaningfully: feat, fix, docs, chore, merge.
- Each commit is scoped to a single logical change, which keeps git bisect and code review efficient.
- Minor gap: a small number of early commits (before the convention was adopted) use generic messages such as "update files" — flagged below as an improvement area.



If CI fails or a reviewer requests changes, the branch returns to the developer for further commits (feedback loop, not shown to avoid clutter).

Figure 2: Code review and CI/CD merge workflow used before any branch reaches develop or main.

4.4 Repository Maintenance Practices

- main is a protected branch — direct pushes are disabled; changes only arrive via reviewed, CI-passing pull requests.
- Releases are tagged (v1.0.0, v1.0.1), giving a citable, immutable reference point for any deployed version.
- Stale feature branches are deleted after merge to keep the branch list readable.

4.5 How Version Control Supports Collaborative Development

Because each service lives in its own folder and each feature in its own branch, multiple contributors work in parallel with minimal merge conflicts. The develop branch acts as a continuous integration point where conflicts surface early, in a lower-stakes environment than main. Pull requests double as a review gate and as documentation of why a change was made, which matters for a project with contributors who may rotate each semester. Tags give disaster-management stakeholders (an end user group named in the problem statement) a stable, versioned reference for which build is in production at any time.

5. Code Testing and Validation

5.1 Test Case Summary

ID	Component	Input	Expected Output	Status
TC-01	ingestion-service	Valid NASA API URL	Returns parsed JSON, writes raw file	Pass
TC-02	ingestion-service	Unreachable API URL	Raises RequestException, logs error	Pass
TC-03	processing-service	Temperature series with one outlier	Anomaly flagged with correct index	Pass
TC-04	backend-api	GET /api/regions/:id/timeseries	200 OK with expected JSON schema	Pass
TC-05	backend-api	Invalid region ID	404 with descriptive error message	Pass
TC-06	frontend	User selects overlay layer	Map re-renders with both layers visible	Pass

5.2 Example Unit Test — Processing Service (pytest)

```
def test_detects_temperature_anomaly():
    series = [21.0, 21.4, 21.1, 34.8, 21.3] # index 3 is an outlier
    result = detect_anomalies(series, threshold_std=2.0)
    assert result.anomaly_indices == [3]
    assert result.anomaly_indices != []
```

5.3 Example Integration Test — Backend API (Jest + Supertest)

```
test('GET /api/regions/:id/timeseries returns 200 with expected shape', async () => {
  const res = await request(app).get('/api/regions/12/timeseries');
  expect(res.statusCode).toBe(200);
  expect(res.body).toHaveProperty('temperature');
  expect(res.body).toHaveProperty('rainfall');
});
```

5.4 Sample Execution Output

```
$ pytest tests/unit/ -v
test_ingestion.py::test_fetch_success PASSED
test_ingestion.py::test_fetch_handles_network_error PASSED
test_processing.py::test_detects_temperature_anomaly PASSED
===== 3 passed in 1.42s =====

$ npm test
PASS tests/integration/regions.test.js
  ✓ GET /api/regions/:id/timeseries returns 200 with expected shape (48 ms)
  ✓ GET /api/regions/:id/timeseries with invalid id returns 404 (21 ms)
Test Suites: 1 passed, 1 total
Tests:      2 passed, 2 total
```

Note: Replace the sample outputs above with your own terminal screenshots once you run the actual test suite against your repository — evaluators expect evidence captured from your specific execution, not illustrative output.

6. Repository Documentation

6.1 README Completeness Checklist

Section	Present?	Notes
Project overview	Yes	Explains purpose and target users clearly.
Installation / setup	Yes	docker compose up steps included.
Usage guide	Partial	Covers the map UI; missing export and filtering walkthrough.
Environment variables	Yes	.env.example is referenced and documented.
API reference	Partial	api-spec.md exists but is not kept fully in sync with routes/.
Contribution guidelines	Missing	No CONTRIBUTING.md; branching convention is undocumented for new contributors.
License	Yes	LICENSE file present at root.

6.2 Suggested Improvements

- Add a CONTRIBUTING.md documenting the Gitflow branching convention, commit message format, and PR checklist for new contributors.
- Generate api-spec.md automatically from route annotations (e.g., OpenAPI/Swagger) so it cannot silently drift from the actual endpoints.
- Expand the usage guide with short walkthroughs for the export and region-filtering features, since these are objectives explicitly called out in the problem statement.
- Add a troubleshooting section covering common Docker Compose issues (port conflicts, missing .env values).
- Add status badges (build passing, test coverage) to the top of README.md for at-a-glance repository health.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Area	Rating	Key note
Repository organization	Good	Clear one-folder-per-service layout; naming conventions consistent.
Code quality	Satisfactory	Solid modularity; inline documentation and type hints need standardizing.
Version control	Good	Gitflow well applied; a few pre-convention commit messages are weak.
Testing	Satisfactory	Core paths covered; error-path and frontend test coverage could expand.
Documentation	Satisfactory	README strong on setup, weaker on usage/API reference and contributor guidance.

7.2 Recommendations

- Add pre-commit hooks (Black/Flake8 for Python, ESLint/Prettier for JS) so style issues are caught before a PR is even opened.
- Introduce a minimum code-coverage threshold in CI (e.g., 80%) to prevent regressions in test coverage over time.
- Adopt a shared logging module across services for consistent log format and easier debugging in production.
- Add a CODEOWNERS file so pull requests automatically request review from the right service owner.
- Automate API documentation generation to keep api-spec.md permanently in sync with backend-api/routes/.
- Retroactively squash or annotate the handful of early, low-information commit messages if repository history is rewritten before public release (only if the branch has not yet been shared/reviewed by others).
- Expand integration tests to cover the export and region-filtering features end-to-end, matching the project's stated objectives.